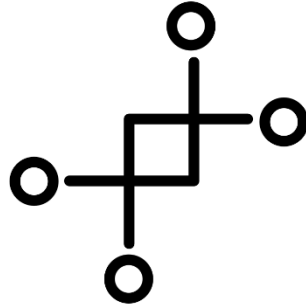




# The pragmatic architect mentorship case study

NETWORK MONITORING APPLICATION

This document describes the combined architecture of a **network monitoring system** that unifies three previously separate views:

1. The **base network monitoring architecture** (probe, Kafka, session storage, UI, PKI).
2. A **tshark-based probe variant**, where the network sensor relies on tshark (Wireshark CLI) to capture and decode traffic.
3. An **updated system architecture** that adds:
  - **Automatic device discovery**.
  - A **device management** capability through the backend and web application.
  - A **shared domain model library (DLL)** reused by the probe, backend and integration console.
  - **Session indexing in Elasticsearch** via **Apache Kafka Connect**.
  - **RBAC (Role-Based Access Control)** for users, implemented with **Keycloak** as Identity Provider (IdP).

The system targets industrial or enterprise environments where passive monitoring, asset visibility, and secure communications are crucial.

---

## 1. Network Monitoring – Technical Architecture Specification

### 1.1. Overview

The network monitoring system is composed of:

- A **passive network probe (sensor)** that observes industrial or enterprise network traffic from a mirror (SPAN) port or TAP.
- A **messaging backbone based on Apache Kafka**, where session events are published.
- A **session processing and storage service** that consumes events from Kafka and persists them in a relational database.
- A **web API and user interface** that expose sessions and devices for querying, filtering and management.
- A **PKI based on EJBCA** that issues X.509 certificates and protects all internal communications using **mutual TLS (mTLS)**.
- A **Keycloak server** that manages user identities and roles, implementing RBAC for human users accessing the UI and API.

The probe extracts metadata from connections (source/destination IP, ports, protocol, timestamps, duration, bytes, etc.) and publishes session events to a Kafka topic in a structured format (JSON/Avro).

A consumer service processes these messages, applies enrichment and validation, and stores them in a database. The UI interacts with the backend API to list and filter sessions and devices.

All communications between internal components (probe → Kafka, backend → Kafka, integration console → backend, etc.) are protected with **mTLS** using certificates issued by EJBCA, while users authenticate and authorize through **Keycloak**.

## 1.2. Main Components

### *Network Probe (Sensor)*

- Lightweight microservice deployed close to the network segment being monitored.
- Uses tshark to capture traffic in promiscuous mode and extract relevant metadata.
- Identifies flows and sessions (IP, TCP/UDP ports, protocol) and emits **session events**.
- Detects **devices** based on observed MAC/IP pairs, ARP, LLDP/CDP or other passive mechanisms and emits **device events**.
- Publishes:
  - Session events to a Kafka topic (e.g. sessions.detected).
  - Device events to another Kafka topic (e.g. devices.detected).
- Uses a certificate issued by EJBCA to connect securely to Kafka via mTLS.

### *Kafka Cluster*

- **Cluster of 3 or more brokers** for high availability.
- Serves as the **event backbone** of the system, decoupling producers (probe) from consumers (backend, integration console, Kafka Connect, analytics).
- Topics:
  - sessions.detected for session events.
  - devices.detected for device discovery events.
- Partitions are configured to allow parallel consumption based on keys (e.g. device ID or IP hash).
- Uses **TLS and mTLS** to authenticate clients and encrypt traffic.

### *Session Processor & API Service*

- Microservice (e.g. Java/Spring Boot, Python/FastAPI, Node.js/Express) that:
  - Consumes session events from sessions.detected.
  - Applies validation, enrichment, deduplication and TTL policies.
  - Persists sessions in a **relational database** (PostgreSQL/TimescaleDB).
- Exposes a **REST (or gRPC) API** for:
  - Listing sessions with filters (time range, IP, protocol, port...).
  - Retrieving details of a specific session.
- Also exposes a **device API** for:

- Listing devices (GET /devices).
  - Creating devices manually (POST /devices).
- Internally follows **clean/hexagonal architecture**:
  - Inbound adapters: REST/gRPC controllers, Kafka consumers.
  - Domain layer: business rules for sessions and devices.
  - Outbound adapters: repositories for database access.

#### *User Interface (Web Application)*

- Web frontend (SPA or server-rendered) that:
  - Authenticates users via Keycloak (OIDC).
  - Consumes the backend API to list and filter sessions.
  - Allows operators to list and manage devices (manual creation, search, etc.).
- The UI sends HTTP requests with a **Bearer token** from Keycloak, and the backend applies **RBAC** based on the user's roles.

#### *EJBCA PKI (CA and RA)*

- EJBCA is deployed as the internal **Certificate Authority**.
- Issues certificates for:
  - Network probes.
  - Kafka brokers.
  - Backend/API services.
  - Integration console.
  - (Optionally) other internal components like Kafka Connect workers.
- Manages the full certificate lifecycle: issuance, renewal, revocation.
- Enables a **zero-trust** communication model enforced by mTLS between services.

#### *Keycloak (Identity and Access Management)*

- Keycloak acts as **Identity Provider (IdP)** and authorization server.
- Manages:
  - Human users (operators, analysts, auditors, administrators).
  - Client applications (web UI, backend, integration console).
  - Roles and group-based RBAC.
- Uses **OpenID Connect (OIDC)** and **OAuth2**:
  - UI redirects users to Keycloak for login.
  - Keycloak returns ID Token and Access Token (JWT) containing user identity and roles.
  - Backend validates the token and applies RBAC.

### **1.3. Architectural Style and Patterns**

- **Microservices and event-driven architecture:**

- Probe, backend, integration console, Kafka Connect, UI and identity services are separate units.
- Communication between services is mostly asynchronous via Kafka.
- **Publisher–Subscriber** pattern:
  - Probe publishes to Kafka topics.
  - Backend, integration console and Kafka Connect subscribe as independent consumers.
- **Clean/Hexagonal architecture** per service:
  - Domain logic isolated from technical details (Kafka, DB, HTTP).
  - Adapters for I/O at the edges.
- **RBAC and external IdP:**
  - Authentication delegated to Keycloak.
  - Backend enforces authorization based on roles and claims in JWT tokens.

## 1.4. Technology Stack

- **Probe:**
  - tshark for packet capture and decoding.
  - Wrapper in C, Go or Python to parse tshark output and publish events to Kafka.
  - Shared domain DLL for Session and Device entities.
- **Messaging:**
  - Apache Kafka with Schema Registry for Avro/JSON schemas.
  - Topics sessions.detected, devices.detected and others as needed.
- **Backend/API:**
  - Java/Spring Boot, Python/FastAPI or Node.js/Express for REST/gRPC.
  - Kafka client library (Kafka Streams, native clients, etc.).
  - PostgreSQL/TimescaleDB for sessions and devices.
- **UI:**
  - SPA (React/Angular/Vue) or server-rendered UI.
  - OIDC integration with Keycloak.
- **Security:**
  - EJBCA-based PKI for mTLS.
  - Keycloak for user authentication and RBAC.

## 1.5. Security Architecture (mTLS and PKI)

- **Service-to-service security:**
  - All internal traffic (probe ↔ Kafka, backend ↔ Kafka, integration console ↔ backend) uses mTLS.
  - Clients and servers present certificates issued by EJBCA.

- Kafka configured with `ssl.client.auth=required` and ACLs bound to certificate identities.
- **User security:**
  - Users authenticate in Keycloak.
  - UI obtains tokens from Keycloak and forwards access tokens to the backend.
  - Backend validates signature, expiry and issuer, and enforces RBAC.

## 1.6. Deployment and Scalability

- **Kafka:**
  - Multiple brokers and partitions for high throughput and fault tolerance.
- **Backend:**
  - Horizontally scalable replicas behind a load balancer.
- **Database:**
  - Scaled vertically and/or with replicas.
  - Partitioning or time-based tables for large session volumes.
- **Probe:**
  - Deployed per relevant network segment.
  - Stateless apart from configuration.

## 1.7. Extensibility and Future Features

The design is modular and allows future extensions:

- Additional analytics services consuming from Kafka.
- Integration with SIEM or SOAR tools.
- Advanced anomaly detection and protocol fingerprinting.
- Export of data to other systems via new Kafka connectors.

# 2. Network Monitoring Architecture (tshark Sensor)

This section focuses specifically on the **tshark-based sensor variant**, which replaces other capture mechanisms with tshark while keeping the rest of the pipeline intact.

## 2.1. tshark-Based Probe

- Runs tshark on a network interface connected to SPAN/TAP.
- Uses capture filters (-f) and display filters (-Y) to focus on relevant traffic.
- Example command:

```
sudo tshark -i eth0 -l -T fields \
-e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport \
-e frame.time_epoch -e ip.proto
```

- -l ensures line-buffered output for real-time streaming.
- -T fields + -e selects specific fields (IPs, ports, timestamps, protocol).
- Additional fields (MAC address, VLAN, LLDP/CDP details, DNS names) can be added as needed.

A wrapper process:

- Reads stdout from tshark line by line.
- Parses each record and maps it to **Session** and **Device** entities from the shared domain DLL.
- Publishes events to Kafka in a consistent schema.

## 2.2. Data Ingestion Pipeline (tshark → Kafka)

- The probe is responsible for building well-formed domain entities and sending them directly to Kafka.
- Alternatively, pipelines like `tshark | nc | kafka-console-producer` can be used for prototyping.
- In production, a dedicated thin service handles:
  - Parsing.
  - Validation.
  - Serialization into Avro/JSON.
  - Publication to Kafka topics (`sessions.detected`, `devices.detected`).

## 2.3. Stream Processing and Downstream Components

Once in Kafka:

- The **backend** consumes events and persists them.
- **Kafka Connect** consumes session events and indexes them into Elasticsearch.
- Other consumers (for analytics, alerting, etc.) can be added without modifying the probe.

The key design goal is that replacing the capture engine with tshark does **not** impact downstream components, as long as the schema and contracts remain stable.

## 2.4. Security and Trust

- The probe uses a client certificate from EJBCA.
- Kafka validates the certificate and authorizes access based on ACLs.
- Sessions and devices events are signed indirectly by virtue of being transported over mTLS.

## 2.5. Summary of the tshark Variant

- tshark is used for packet capture and decoding.
- A thin wrapper converts tshark output into structured domain entities.

- Events are published to Kafka, where the rest of the architecture consumes them without changes.
- 

### 3. Network Monitoring System Architecture

This section adds **device discovery**, **web-based device management**, a **shared domain DLL** and **session indexing in Elasticsearch via Kafka Connect**.

#### 3.1. Extended Architecture Overview

The extended architecture retains the core components (probe, Kafka, backend, UI, PKI) and introduces:

- **Automatic device discovery** from network observations.
- A **device inventory** maintained by the backend.
- A **shared domain model DLL** used by probe, backend and integration console.
- **Elasticsearch** as an additional storage engine for session search and dashboards.
- **Kafka Connect** as the standard integration mechanism from Kafka to Elasticsearch.

#### 3.2. Device Discovery and Kafka Integration

The probe:

- Observes MAC/IP pairs, ARP exchanges, LLDP/CDP and other discovery-related traffic.
- Builds **Device** entities with:
  - MAC address.
  - Associated IP addresses.
  - Hostname (if known).
  - Optional metadata (interface, VLAN, switch port, etc.).
- Publishes Device events to the devices.detected Kafka topic.

#### 3.3. Backend Device Management and Reuse of Insertion Logic

The backend provides a **unified device management model**:

- **Device API:**
  - GET /devices → list devices.
  - POST /devices → create a device.
- **Web UI:**
  - Offers a device list view (queries GET /devices).
  - Allows manual device creation (invokes POST /devices).

- **Integration console:**
  - Consumes events from devices.detected.
  - Uses the shared domain DLL to deserialize Device entities.
  - Forwards them to the backend via POST /devices.

This ensures that **both manual and automatic device creation:**

- Pass through the same backend validation and business logic.
- Respect the same rules for uniqueness and consistency.
- Store devices in a single, authoritative inventory in the database.

### 3.4. Shared Domain Model DLL

A **shared DLL** defines core domain entities:

- Session with fields for IPs, ports, timestamps, protocol, bytes, etc.
- Device with MAC, IPs, hostname and optional attributes.
- Supporting types and enumerations.

This DLL is referenced by:

- The **probe**, to create domain entities before publishing them.
- The **backend**, for a consistent domain model in APIs and persistence.
- The **integration console**, consuming devices.detected and interacting with the backend.

Trade-off (intentional for the learning context):

- Sharing a DLL creates coupling between services.
- However, it guarantees schema alignment and reduces mapping code.

### 3.5. Sessions to Elasticsearch via Kafka Connect

Sessions are not only stored in the relational database but also indexed in **Elasticsearch** for advanced querying and dashboards.

- Session events are produced to sessions.detected by the probe.
- **Kafka Connect** runs an **Elasticsearch sink connector** configured with:
  - topics=sessions.detected
  - connector.class=io.confluent.connect.elasticsearch.ElasticsearchSinkConnector (or similar)
  - connection.url=http://<elasticsearch-host>:9200
  - Additional settings for index naming, retries, etc.

The typical interview/mentoring question to the student architect is:

“How can we send the sessions from Kafka to Elasticsearch in a scalable and maintainable way?”

The expected answer is:

“By using an **Apache Kafka Connect Elasticsearch sink connector** that reads from the Kafka topic and writes into Elasticsearch.”

### 3.6. Suggested Architecture Exercises

The following exercises help the student architect approach the design of this application.

#### 1. Architecture and Deployment Diagrams

- Draw a **component diagram** showing:
  - Probe (tshark-based sensor).
  - Kafka cluster.
  - Backend/API.
  - Integration console.
  - Kafka Connect.
  - Database.
  - Elasticsearch.
  - Web UI.
- Draw a **deployment diagram** detailing:
  - Where each component runs (VMs, containers, Kubernetes).
  - Network zones and boundaries.

#### 2. Architecture Decision Records (ADRs)

- Write ADRs documenting decisions such as:
  - Using Kafka as the event backbone instead of direct HTTP.
  - Using a shared DLL for the domain model.
  - Using Kafka Connect for Elasticsearch integration.
- Each ADR should include context, decision, alternatives considered and consequences.

#### 3. Alternative Options and Trade-Offs

- Analyse alternatives to Kafka, Elasticsearch, and the shared DLL.
- Discuss pros and cons of each alternative in terms of complexity, performance and maintainability.

#### 4. Quality Attributes and Metrics

- Identify critical quality attributes: scalability, reliability, performance, security.
- Define measurable metrics, for example:
  - Maximum acceptable end-to-end latency from packet capture to Elasticsearch indexing.

- Availability targets for Kafka and the backend.
  - Error budgets and alerting thresholds.
- 

## 4. Identity and Access Management (RBAC with Keycloak)

The architecture includes a **role-based identity and access management system** implemented with **Keycloak**.

### 4.1. Role of Keycloak in the Architecture

- **Keycloak** manages:
  - Human users (operators, analysts, auditors, administrators).
  - Client applications (web UI, backend, integration console).
  - Roles and group membership.
  - Authentication flows (login, optional MFA, password recovery).
- The **web frontend** integrates with Keycloak using **OIDC**:
  - The user authenticates against Keycloak.
  - Keycloak issues an **ID Token** and an **Access Token (JWT)** that includes the user's roles.
- The **backend/API**:
  - Receives requests with a Bearer token in the Authorization header.
  - Validates the token signature, audience, issuer and expiry.
  - Extracts identity and roles from token claims.
  - Applies **RBAC** to protect:
    - Session endpoints (/sessions, /sessions/{id}).
    - Device endpoints (/devices, /devices/{id}).
    - Administrative endpoints.

This model complements the use of **EJBCA + mTLS**:

- EJBCA and mTLS protect **service-to-service** communication.
- Keycloak handles **user authentication and authorization** for the UI and any user-facing APIs.

### 4.2. Role Model and Permissions

A potential role model is:

- **ROLE\_ADMIN**  
Full administration of the system: configuration management, device lifecycle operations, connector configuration (e.g. Kafka Connect → Elasticsearch), user and role delegation.

- **ROLE\_ANALYST**  
Read access to sessions and devices, ability to filter, search and export data. Cannot delete devices or change critical configuration.
- **ROLE\_AUDITOR**  
Strict read-only access, focused on compliance and audit. No modifications allowed.
- **ROLE\_INTEGRATION**  
Service role for the **device integration console**:
  - May create or update devices based on devices.detected events.
  - Cannot perform UI-level operations or access interactive analyst features.

Example authorization mapping:

- GET /sessions, GET /devices → allowed for ROLE\_ANALYST and ROLE\_AUDITOR.
- POST /devices → allowed only for ROLE\_ADMIN and ROLE\_INTEGRATION.
- Administrative endpoints (configuration, connector management) → restricted to ROLE\_ADMIN.

### 4.3. Backend–Keycloak Technical Integration

The backend:

1. Trusts Keycloak as the **OIDC provider** and obtains its **public keys** (JWKS endpoint) to validate JWT signatures.
2. Extracts from the Access Token:
  - User identity (sub, preferred\_username).
  - Roles (e.g. realm\_access.roles or resource\_access).
3. Uses middleware, filters or interceptors to check roles **before** executing endpoint logic.

For non-human actors like the integration console, the system can use:

- **Client Credentials Grant** in Keycloak to obtain an Access Token with ROLE\_INTEGRATION.
- Combination of this token with **mTLS** between services to achieve a strong security posture.

### 4.4. Suggested Exercises on RBAC and Keycloak

As part of the architecture mentorship, the following Keycloak-oriented exercises can be proposed:

1. **Design the Role Model**  
Document what each type of user needs to do and map these needs to Keycloak roles.

## 2. Define Authentication Flows

Draw a sequence diagram showing how the user logs in via the UI, obtains tokens from Keycloak and calls the backend with an Access Token.

## 3. Specify Endpoint Protection Policies

Build a matrix that maps endpoints (/sessions, /devices, admin operations) to the roles that can access each of them.

## 4. ADR for Choosing Keycloak

Write an ADR explaining why Keycloak was chosen over alternatives (IdentityServer, Auth0, Cognito, etc.), listing pros and cons.

---

# 5. Exercises: Introduction to Software Architecture

This architecture document can be used directly as a **case study** in *The Pragmatic Architect Mentorship Program – Module 1: Introduction to Software Architecture*. Below are suggested exercises that connect the network monitoring system with the main ideas from Module 1: **Architectural Mindset, Quality Attributes, Fitness Functions and Metrics**.

## 5.1. Architectural Mindset vs Developer Mindset

### 5. Scope & Focus Exercise

Using this network monitoring system as context, write two short design notes:

- One from the perspective of a **developer** tasked with implementing the POST /devices endpoint.
- One from the perspective of an **architect** defining how device management should work end-to-end (probe → Kafka → integration console → backend → UI). Highlight differences in scope (local vs system-wide) and concerns (implementation details vs overall flow and boundaries).

### 6. Time Horizon & Trade-Offs

Identify three design decisions in this architecture (e.g. using Kafka, sharing a domain DLL, using Kafka Connect for Elasticsearch). For each decision:

- Describe the **short-term benefit** (for current sprint/release).
- Describe potential **long-term impact** (maintenance, evolution, performance, team autonomy).
- Explain how an **architect** would consciously balance these trade-offs versus how a **developer** might see them.

### 7. Business-Driven Architectural Thinking

Assume three different business drivers for this system:

- A regulated critical-infrastructure operator (security & reliability first).
- A SaaS startup selling network visibility (time-to-market & scalability first).

- An internal tool for a small IT team (cost & simplicity first). For each scenario, list **which quality attributes become top priority** and describe **two architectural changes** you would make to adapt the current design (e.g. features you would simplify, patterns you would add/remove).
8. **Adopting an Architectural Mindset While Coding**  
 Choose one concrete feature (e.g. “list sessions with filters” or “create device manually from UI”). As if you were a developer implementing it:
- Sketch a simple **class/component diagram** for your implementation.
  - Annotate it with notes about how you would keep the code **adaptable to future changes** (e.g. abstractions, interface boundaries, avoiding premature technology lock-in).
  - Explicitly write down at least **two trade-offs** you considered (e.g. simplicity vs extensibility, performance vs readability) and why you chose one.

## 5.2. Quality Attributes and Architecture Characteristics

### 5. Identify and Prioritise Quality Attributes

For this monitoring system, list at least **8 quality attributes** (performance, security, scalability, maintainability, reliability, usability, interoperability, etc.). Then:

- Work with a hypothetical stakeholder (role-play) to **rank them by importance** for this project.
  - For the top 3 attributes, explain **which current design decisions** support them and **where the architecture is weak or ambiguous**.
6. **Scenario–Response Quality Attribute Workshop**  
 Define **3–5 quality attribute scenarios** in the format *Stimulus – Environment – Response* (e.g. “Under peak load of N probes, the system should still return session queries in < 1 second for 95% of requests”). For each scenario:
- Map which components and interactions are involved.
  - Check whether the current architecture description is sufficient to meet the scenario. If not, propose **concrete design adjustments** (e.g. additional caching, read replicas, circuit breakers).
7. **Trade-Off Matrix Exercise**  
 Build a simple **trade-off matrix** where rows are quality attributes (performance, security, maintainability, scalability, etc.) and columns are key decisions (Kafka, shared DLL, Kafka Connect, Keycloak, EJBCA, microservices decomposition). For each cell, mark whether that decision **helps (+)**, **hurts (–)**, or is **neutral (~)** for that attribute and write a one-sentence justification.

## 5.3. Fitness Functions for the Network Monitoring Architecture

### 8. Define Architectural Fitness Functions

Propose at least **5 fitness functions** for this system, covering:

- Performance (e.g. response time under load for /sessions).

- Security (e.g. all endpoints require valid JWT; no critical vulnerabilities in dependencies; all internal calls use TLS).
- Structural rules (e.g. UI layer cannot directly access the database; probe code cannot import backend-only packages).
- Reliability (e.g. service health checks and error rates within thresholds). For each fitness function, specify:
  - What is measured.
  - How it is measured (tool or script).
  - The **pass/fail threshold**.

#### 9. **Convert Architecture Rules into Automated Checks**

Take 2–3 important architectural rules from this document (for example, “Only the backend can write to the database” or “All service-to-service calls must use mTLS”). Design how you would implement them as **automated fitness functions** integrated into CI/CD or infrastructure:

- Which tool/technology would you use (e.g. ArchUnit, policy as code, infrastructure tests)?
- How often would they run?
- What should happen when they fail (block the pipeline, raise an alert, etc.)?

### 5.4. Metrics and Tools

#### 10. **Define a Metrics Dashboard for This System**

Based on the metrics taxonomy (performance, security, maintainability, reliability, etc.), design a **dashboard** for this network monitoring solution:

- Choose **3–5 key metrics** per quality attribute category.
- Specify where each metric comes from (Prometheus, APM tool, SonarQube, security scanner, etc.).
- Define target values or ranges for each metric.

#### 11. **Tooling Map Exercise**

Create a table that maps:

- Quality attributes → metrics → tools.  
Example row: *Performance* → *95th percentile response time for /sessions* → *JMeter in CI + APM in production*.  
Do this for at least performance, security, maintainability and reliability.

#### 12. **Interpreting Metrics and Prioritising Technical Work**

Imagine you receive the following metric trends over the last 3 sprints:

- Average response time for /sessions increased from 150ms to 350ms.
- SonarQube maintainability index dropped by 10 points in the backend service.
- Error rate in production remained stable.
- Security scanner reports 1 new high-severity vulnerability in a transitive dependency. Write a short **architect’s report** that:
- Interprets these results in context.

- Prioritises actions (what to fix first and why).
- Suggests architectural or process changes to prevent similar issues.

### 13. Linking Module 1 to the Case Study

Finally, write a one-page reflection connecting Module 1 concepts to this architecture:

- How does this system illustrate the **difference between developer and architect thinking**?
- Which **architectural mindset characteristics** (holistic view, trade-off analysis, business alignment, forward-thinking, communication) are most clearly visible in the design decisions here?
- How do **quality attributes, fitness functions and metrics** show up concretely in this case study?

## 6. Exercises: Software Architecture Design

This module focuses on designing modular systems, choosing appropriate architectural styles, making and documenting architecture decisions, and creating useful diagrams. The exercises below cover **components, architectural styles, decision records, and diagramming**.

### 6.1. Components

- **Identify and classify components:** Take a hypothetical project description (for example, an online bookstore or a ride-sharing app). List at least four logical components (e.g., user interface, order processing, inventory, payment). For each component, specify its type (presentation, business logic, or data) and describe its primary responsibilities. Explain how these components should interact with each other while minimizing coupling and maintaining high cohesion.
- **Analyze and reduce coupling:** Given a simple system design or code snippet (you can create a small example or use an existing one) where components are tightly coupled (for example, the user interface code directly queries the database), identify the coupling issues. Propose design changes to decouple components (such as introducing interfaces, abstractions, or a service layer) and explain how your changes improve modularity and maintainability.
- **Refactor a monolith:** Take a small monolithic application (or a detailed description of one) and outline how to decompose it into independent components or modules. For instance, break a single-class “OrderManager” into separate classes or services like OrderService, PaymentService, and InventoryService. Describe the responsibilities of each new component and how data should flow between them. Optionally, sketch a simple diagram showing the components and their relationships.

## 6.2. Architectural Styles

- **Style selection case study:** Given different project scenarios, choose the most suitable architectural style and justify your choice. For example, consider: (a) A content-heavy website with unpredictable traffic (might favor microservices or serverless), (b) A simple internal tool for a small team (could be a monolithic app), (c) A real-time multiplayer game server (maybe event-driven or client-server). For each scenario, explain why a particular style (e.g., monolithic, client-server, microservices, event-driven) is a good fit, and mention any trade-offs.
- **Compare styles and patterns:** Create a comparative table or write a short report comparing two or three common styles (e.g., monolithic vs. microservices vs. layered architecture). Include their strengths and weaknesses, typical use cases, and real-world examples. Then, for each chosen style, identify one or two associated design patterns (for instance, the MVC pattern for layered web apps, or the Circuit Breaker pattern in microservices). Describe how each pattern addresses a specific challenge in that style.
- **Style in practice:** Examine a well-known software system (you can pick something like an e-commerce platform, social media app, or banking system) and research what architectural style it uses or uses historically. Write a brief case study summarizing that system's architecture style, why it was chosen, and how it matches the system's requirements (e.g., scalability, maintainability). Discuss whether this style would still be appropriate if the system's scale or requirements changed.

## 6.3. Architectural Decisions (ADR)

- **Write an ADR:** Pick an important decision in a software project (for example, choosing a database type, a frontend framework, or a deployment environment). Write an Architecture Decision Record following a standard format (including context, decision, alternatives, and consequences). For instance, decide between using a relational database (like PostgreSQL) or a NoSQL database (like MongoDB) for a high-read application. Document why you made the decision you did and what impact it will have on the system (performance, scalability, complexity).
- **Analyze a scenario:** Read a short case scenario of a past project (create one or find a real example) where architects made several decisions (e.g., "We switched from monolithic to microservices, we chose REST over gRPC, we opted for AWS for hosting"). Identify at least three distinct decisions in that scenario. For each decision, draft a concise ADR entry summarizing the context and rationale. This exercise helps practice distilling decisions into clear records.
- **Create an ADR template:** Research existing ADR templates (such as Michael Nygard's ADR pattern) and then draft your own template or checklist. Include sections like "Status," "Context," "Decision," "Consequences," etc. Use your

template to write an ADR for a hypothetical decision, such as introducing a new caching layer (e.g., “We will use Redis to cache session data to improve performance.”). Ensure your ADR clearly communicates the reasons and implications of the decision.

## 6.4. Diagramming in Software Architecture

- **Draw a component diagram:** Choose a sample system (e.g., a food delivery app or a library management system) and describe its major parts. Use a diagramming tool (like draw.io, Lucidchart, or even pen and paper) to draw a component (or package) diagram showing key components and their relationships. For example, include components such as a mobile app, API gateway, order service, user service, and database. Show the interactions or dependencies between them. Explain how the diagram illustrates the high-level design.
- **Create a deployment diagram:** Given a distributed application scenario (for instance, a three-tier web application with load balancers, web servers, application servers, and database servers), sketch a deployment diagram. Label the nodes (machines or containers), components deployed on each node, and the network connections. For example, show two web server instances behind a load balancer, an app server cluster, and a replicated database. Describe how this deployment supports scalability and fault tolerance.
- **Interpret and critique an architecture diagram:** Find or draw an example of an architecture diagram (it could be a UML diagram, AWS architecture diagram, etc.). Write a short explanation of what each part of the diagram represents. For example, explain that “Component A is responsible for user authentication” or “Service B communicates with Service C via REST API”. Then critique the diagram’s clarity: Is it missing any components? Is the layout easy to follow? Suggest improvements or additional diagrams (like adding a data flow or sequence diagram) that could help other team members understand the system better.

## 7. Exercises: Implementation and Maintenance

This module covers translating architectural designs into code, ensuring quality through testing, deploying software to production, and ongoing monitoring. The exercises below address **implementation practices**, **testing strategies**, **deployment automation**, and **monitoring & maintenance**.

### 7.1. Implementing the Architecture in Code

- **Skeleton implementation:** Based on a given architecture diagram or description (for example, a layered architecture for a note-taking app), write basic code stubs or class definitions for each component in your preferred programming language. Ensure that the code reflects the architecture’s separation of

concerns. For instance, create classes or modules like `UserInterface`, `BusinessLogic`, `DataAccess`, and show how they interact (e.g., the UI class calls methods in the business logic class). Include comments or docstrings to explain each part's role.

- **Refactor to match architecture:** Take a snippet of code (either one you write or find an example) that mixes concerns (such as database calls inside a UI handler). Refactor the code to enforce the intended architecture. For example, move database operations into a separate repository or service class, and keep the UI layer only responsible for presentation. Comment on how your refactoring improves clarity and adherence to the architecture design.
- **Performance considerations:** Imagine your system design includes a performance-critical component (e.g., a caching layer or a messaging queue). Write a short plan or pseudocode illustrating how you would implement that component. Discuss any performance optimizations you would include (for instance, using asynchronous processing, implementing caching, or pooling resources). Explain how you would measure the performance impact (e.g., using profiling tools or load tests).

## 7.2. Software Testing and Quality Assurance

- **Design a test strategy:** For a small multi-component system (e.g., a REST API with a database and a frontend), outline a testing strategy. Define what unit tests, integration tests, and system tests you would write. For instance, list critical unit test cases for core functions, describe integration tests between the API and database, and plan a system test scenario covering a user story. Explain which testing tools or frameworks (like JUnit, pytest, Selenium) you might use and why.
- **Write sample tests:** Pick one component from your system (such as the data access layer) and write actual unit test code for it. For example, if you have a function that calculates discounts, write tests for different input cases. Show how mocks or stubs might be used for external dependencies (e.g., mocking a database). Include assertions that verify correct behavior. If possible, demonstrate running the tests and checking coverage or results.
- **Fault injection and quality:** Simulate a failure in the system (for example, have a component throw an exception, or break the database connection). Observe how your existing tests respond. Write a brief report on what failures were caught and what went unnoticed. Then improve the tests or code: add a new test to catch an edge case you missed, or handle the failure in code (e.g., with try/catch and a fallback). This exercise emphasizes writing tests that cover real-world error conditions.

### 7.3. Deployment and Production Release

- **Plan a CI/CD pipeline:** Draft a continuous integration/continuous deployment (CI/CD) pipeline for your project. Outline steps such as code checkout, automated build, running tests, and deployment to a staging or production environment. Specify which tools you would use (e.g., GitHub Actions, Jenkins, GitLab CI, Docker). For instance, describe a workflow file that triggers on commits to the main branch and runs tests before deploying to a test server.
- **Compare deployment strategies:** Consider three deployment strategies — **Blue-Green Deployment**, **Canary Releases**, and **Rolling Updates**. For a given scenario (e.g., releasing a new version of an e-commerce website), explain how each strategy would work step by step. List the advantages and risks of each method. For example, explain that a blue-green deployment keeps the old version running while the new version is deployed in parallel, which minimizes downtime.
- **Automate deployment tasks:** Choose one aspect of deployment automation to practice. For example, write a Dockerfile and docker-compose setup that packages your application and its database. Or write a script (shell or a CI tool) that connects to a server and pulls a new version of the code. Explain how this automation helps reduce manual errors. Optionally, if you have access, try deploying a simple app to a free cloud service (like Heroku, AWS Free Tier, or Azure App Service) and document the process.

### 7.4. Monitoring and Maintenance of the Architecture

- **Set up monitoring metrics:** Identify key performance indicators (KPIs) for your system (such as response time, error rate, CPU usage). Choose monitoring tools or services (like Prometheus/Grafana, New Relic, or built-in cloud monitors). Create a plan or configuration showing how you would collect these metrics. For instance, describe setting up an alert that notifies the team if average request latency exceeds a threshold for 5 minutes.
- **Analyze logs and alerts:** Given a sample log file or monitoring dashboard output (you can simulate this or use example data), look for signs of issues (e.g., error messages, slow queries, high memory usage spikes). Write a short incident report identifying what went wrong and which part of the architecture is affected. Suggest fixes or improvements (for example, adding caching, fixing a bug, or increasing a resource limit).
- **Architecture evolution plan:** Imagine the system needs to evolve due to new requirements (e.g., supporting ten times more users or adding a mobile app). Draft a maintenance plan that addresses this evolution. Include how you would update architecture documentation (diagrams, ADRs), refactor code or components, and communicate changes to the team. Describe a process for regularly reviewing and updating the architecture (such as scheduled architecture review meetings or versioning the design documents).

## 8. Exercises: Soft Skills and Business Strategy

This module emphasizes the human and strategic side of architecture: effective communication, leadership, and aligning architecture with business goals. The exercises below cover **communication and leadership**, as well as **strategic planning tools** like Wardley Mapping and Event Storming.

### 8.1. Soft Skills

- **Communication case study:** Prepare to explain a complex technical concept (like microservices or database sharding) to a non-technical stakeholder (such as a project manager or client). Role-play or write a one-page summary focusing on business benefits rather than technical details. Use analogies or simple diagrams if helpful. For example, compare microservices to a fleet of delivery trucks (independent but coordinated) rather than giving low-level engineering details.
- **Team leadership scenario:** You are the lead architect and two senior developers disagree on how to implement a feature. Plan and describe how you would facilitate a resolution. For instance, you might organize a short meeting, encourage each person to outline their reasoning, and steer the discussion towards shared project goals. Write a dialogue script or bullet points showing how you ensure all voices are heard and a final decision is made collaboratively.
- **Conflict resolution exercise:** Create a fictional conflict situation in a software team (such as one team member feeling another is not following coding standards). Write a brief reflection or conversation script demonstrating how you would address the issue. Practice using active listening, empathy, and clear communication. For example, show how you would acknowledge the concerns of the team member and work with the team to agree on coding guidelines moving forward.

### 8.2. Architecture and Business Strategy

- **Alignment with business goals:** Given a hypothetical company strategy (for example, “Expand to 3 new markets in 6 months” or “Reduce operating costs by 20%”), propose how the software architecture should support these goals. Write a brief plan outlining architectural initiatives that align with the business objective. For example, to scale to new markets, you might suggest adding multi-regional support (architectural scaling, localized data stores) and define metrics (like deployment frequency or system uptime) that measure progress.
- **Define North Star Metrics:** For a sample product (such as a subscription-based video service), identify 2–3 North Star Metrics that align engineering work with business value. Explain why each metric is crucial. For instance, “Monthly Active Users” might reflect user engagement, and “Average Response Time” reflects

user satisfaction. Describe how architectural decisions (like adding caching or load balancing) could directly impact these metrics.

- **Wardley Mapping exercise:** Choose a product or service (e.g., an online grocery delivery app) and create a simple Wardley Map. Identify the user needs (like “Order groceries”), list the value chain activities (customer interface, order processing, payment processing, infrastructure), and place them on the evolution axis (genesis to commodity). Use the map to identify strategic moves: for example, if a component is at the commodity stage, you might suggest outsourcing or automating it. Provide a sketch or description of your map and the insights gained (such as spotting an overinvested custom component).
- **Event Storming workshop:** Conduct an event storming session for a specific business process. Pick a scenario like “User signs up and places an order.” Use sticky notes or a digital board to identify the domain events (e.g., *UserRegistered*, *OrderPlaced*, *PaymentProcessed*), commands (like *RegisterUser*, *CreateOrder*), and aggregates or bounded contexts (such as User, Order, Payment). Document the flow of events from start to finish. Write a summary of what you discovered, such as missing business rules or simplifications (e.g., realizing that shipping address capture belongs in the Order domain, not the User domain).
- **Risk management planning:** Identify potential technical and business risks for a project’s architecture (for example, “Single database could become a bottleneck” or “Regulatory compliance requirement is unclear”). For each risk, list mitigation strategies. Write a risk register table with columns for *Risk*, *Impact*, *Likelihood*, and *Mitigation*. For instance, if a single point of failure is a risk, mitigation could be introducing database replication or a backup system. This helps practice proactively thinking about how architecture decisions affect business continuity.